

Universidad Nacional de Ingeniería
Facultad de Ingeniería Industrial y de Sistemas
Escuela de Ingeniería de Sistemas



SI 807 Sistema de Inteligencia de Negocios “U”

"Informe Técnico Integrado: Arquitectura de Datos en la Nube y Solución BI para Telecomunicaciones"

Proyecto: Análisis de Eficiencia Operativa en SOTs y Contratas

Integrantes:

- | | |
|--------------------------------------|-----------|
| ● Callupe Pardo, Yoselyn Patricia | 20190267H |
| ● Garro Oré, Willian Jesús | 20172115E |
| ● Nuñez Poma, Robert Gianpiero Jesus | 20202084E |

Docente:

- Aradiel Castañeda, Hilario
- Garcia Atuncar, Fernando

Lima – Perú

2025

1. Resumen ejecutivo de la solución.....	3
2. Contexto organizacional y definición del problema.....	4
2.1. Descripción de la entidad y ámbito operativo.....	4
2.2. Alineación estratégica.....	4
2.3. Diagnóstico de la situación actual (As-Is).....	5
2.4. Objetivos del proyecto BI.....	5
3. Arquitectura cloud y justificación de servicios (GCP).....	6
3.1. Diagrama conceptual del flujo de datos.....	6
3.2. Análisis de cómputo: Evolución de la infraestructura.....	7
4. Estrategia de datos: Diseño del data lake y gobierno.....	7
4.1. Estructura de buckets y zonas.....	8
5. Ingeniería de datos: Pipeline ETL con PySpark.....	8
5.1. Análisis del script de transformación.....	9
6. Modelado dimensional y data warehouse (BigQuery).....	10
6.1. Tabla de hechos: fact_sot.....	10
6.2. Dimensiones: dim_contrata.....	11
7. Seguridad, IAM y gestión de costos (FinOps).....	11
7.1. Gestión de identidades (IAM).....	11
7.2. Seguridad en aplicación.....	12
7.3. Estrategia FinOps (Clusters Efímeros).....	12
8. Metodología de desarrollo y versionamiento (GitHub).....	12
9. Conclusiones técnicas.....	12

1. Resumen ejecutivo de la solución

El presente informe técnico detalla la implementación de una arquitectura de datos moderna desplegada sobre Google Cloud Platform (GCP). El objetivo central del proyecto fue resolver la latencia en el procesamiento de Solicitudes de Orden de Trabajo (SOTs) y la evaluación de desempeño de las contratas operativas.

Ante el desafío de procesar volúmenes masivos de datos no estructurados y transformarlos en inteligencia accionable, se optó por una transición de sistemas on-premise hacia una arquitectura Cloud-Native. La solución orquesta un flujo de datos ETL utilizando Google Cloud Storage como Data Lake, Dataproc (Apache Spark) para el cómputo distribuido de alto rendimiento, y BigQuery como Data Warehouse empresarial.

La implementación final logró reducir los tiempos de procesamiento de horas a minutos mediante la optimización de clústeres efímeros y scripts de PySpark vectorizados, garantizando la integridad de los datos financieros y operativos bajo estrictos estándares de seguridad y gobernanza.

2. Contexto organizacional y definición del problema

2.1. Descripción de la entidad y ámbito operativo

La organización objeto de estudio es Claro Perú (América Móvil Perú S.A.C.), subsidiaria clave de América Móvil y actor dominante en el ecosistema de telecomunicaciones de Latinoamérica. En un mercado saturado donde la commoditización de los servicios de voz y datos es evidente, la diferenciación estratégica reside en la excelencia operativa y la calidad de la experiencia del cliente (CX).

El alcance técnico de este proyecto se circunscribe a la Gerencia de Operaciones y Mantenimiento (Planta Externa). Esta unidad es responsable de la gestión del ciclo de vida de las Solicitudes de Orden de Trabajo (SOT), las cuales representan la materialización del servicio: desde el aprovisionamiento de nuevas tecnologías (Fibra Óptica/HFC) hasta el mantenimiento correctivo post-venta. Dada la capilaridad nacional de la red, la supervisión eficiente de las empresas contratistas (terceros encargados de la ejecución en campo) es crítica para la rentabilidad y la reputación de la marca.

2.2. Alineación estratégica

La implementación de esta arquitectura de Big Data no es un ejercicio puramente tecnológico, sino un habilitador directo de los vectores estratégicos de la corporación:

- **Misión:** "Proporcionar soluciones de telecomunicaciones de clase mundial..."
 - Impacto de la Arquitectura: Al reducir la latencia en el análisis de las SOTs mediante Google Cloud Platform, garantizamos que la toma de decisiones correctivas (ej. reasignación de cuadrillas) ocurra en ventanas de tiempo que minimicen la interrupción del servicio al usuario final.
- **Visión:** "Ser líderes en transformación digital..."
 - Impacto de la Arquitectura: La migración de flujos de trabajo manuales y legados hacia un paradigma Serverless y Distribuido (Spark/BigQuery) constituye un paso tangible hacia la madurez digital, eliminando silos de información.

2.3. Diagnóstico de la situación actual (As-Is)

A pesar de la robustez comercial de la organización, el proceso de inteligencia operativa presentaba deficiencias estructurales críticas que motivaron la reingeniería del sistema:

1. Ineficacia en el Procesamiento de Datos Semi-Estructurados:

Los sistemas transaccionales modernos generan logs y registros en formatos JSON complejos y voluminosos. Las herramientas legadas on-premise (basadas en SQL tradicional y scripts secuenciales) eran incapaces de parsear y transformar esta data a la velocidad requerida, generando cuellos de botella que impedían una visión "Casi en Tiempo Real" (NRT) de la operación.

2. Opacidad en la Detección de Fraudes Operativos:

Uno de los hallazgos más graves era la incapacidad de detectar patrones anómalos en las liquidaciones de las contratas. No existía un mecanismo automatizado para auditar masivamente los estados de las SOTs (ej. órdenes marcadas como "INSTALADA" pero sin tráfico de red, o "RECHAZADA" injustificadamente). Esta carencia de lógica de validación compleja (ahora implementada en PySpark) derivaba en fugas de ingresos significativas.

3. Silos de Información y Falta de Escalabilidad:

La analítica dependía de reportes estáticos y retrospectivos. La ausencia de un Data Warehouse centralizado impedía cruzar variables críticas como Tiempos de Atención vs. Costos Operativos por región. Además, la infraestructura física carecía de elasticidad para soportar picos de carga (ej. campañas comerciales masivas), resultando en caídas del servicio de reportaría.

2.4. Objetivos del proyecto BI

Para mitigar estos riesgos, se estableció el despliegue de una solución de Inteligencia de Negocios sobre GCP con los siguientes objetivos técnicos:

- **Objetivo general:** Diseñar una arquitectura escalable que automatice el ciclo de vida del dato (ETL), permitiendo a la Gerencia monitorear KPIs críticos como Fraudes y Tiempos de Atención con alta fiabilidad.
- **Objetivos específicos:**

1. Implementar un Data Lake en GCS con zonas segregadas (Raw/Staging/Analytics) para gobernar la ingesta de archivos JSON.
2. Desarrollar pipelines de procesamiento distribuido en Dataproc (Spark) para aplicar reglas de negocio complejas (limpieza de fechas, detección de fraudes) sobre grandes volúmenes de datos.
3. Consolidar una "Fuente Única de la Verdad" en BigQuery, utilizando técnicas de particionamiento para optimizar costos y tiempos de respuesta.

3. Arquitectura cloud y justificación de servicios (GCP)

La arquitectura de la solución se ha erigido sobre un patrón de diseño de procesamiento por lotes (Batch Processing), descartando enfoques de streaming puro (como arquitecturas Kappa) tras evaluar las restricciones de negocio y la naturaleza de los datos. La razón fundamental de esta decisión arquitectónica radica en el ciclo de vida financiero de las órdenes de trabajo (SOTs): las liquidaciones a las empresas contratistas se basan en "cierres operativos" diarios.

En este contexto, la inmediatez del dato es secundaria frente a la integridad y auditabilidad del mismo. Un flujo en tiempo real podría introducir estados transitorios (ej. una SOT que pasa de "Pendiente" a "Instalada" y luego a "Rechazada" en minutos) que complicarían el cálculo de penalidades contractuales. Al optar por un modelo Batch con periodicidad diaria (T+1), garantizamos una ventana de tiempo estática para la limpieza, deduplicación y enriquecimiento de los datos, asegurando que los reportes financieros generados en BigQuery sean consistentes, inmutables y trazables para la Gerencia de Operaciones. Asimismo, este enfoque simplifica la recuperación ante fallos (disaster recovery) mediante la re-ejecución idempotente de los jobs de Dataproc, una capacidad crítica para el cumplimiento de normativas de gobernanza de datos.

3.1. Diagrama conceptual del flujo de datos

El pipeline de datos se estructura en tres etapas lógicas que reflejan la madurez del dato:

1. **Ingesta (Raw Data):** Los sistemas transaccionales vuelcan archivos JSON crudos en el Bucket Regional gs://proyectofinalbi2025-2-datalake.
2. **Procesamiento (Compute Layer):** Un clúster gestionado de Dataproc levanta instancias virtualizadas para ejecutar trabajos de Spark, leyendo desde GCS y realizando transformaciones en memoria.
3. **Almacenamiento analítico (Serving Layer):** Los datos procesados se materializan en **BigQuery** dentro del dataset sot_analytics, habilitando consultas SQL de alta velocidad.

3.2. Análisis de cómputo: Evolución de la infraestructura

Durante las fases de desarrollo (PC3 y PC4), iteramos sobre la configuración del hardware para equilibrar el Trade-off entre Costo y Rendimiento.

- **Fase de reparación (cluster-fix-v2):** Inicialmente se utilizaron máquinas e2-standard-2. Si bien son económicas, presentaron cuellos de botella de memoria (OOM - Out Of Memory) al intentar realizar operaciones de join complejas sin optimización de shuffle.
- **Fase de producción (cluster-masivo-final):** Para la carga final optimizada de 1 millón de registros, escalamos vertical y horizontalmente hacia máquinas n1-standard-4 (4 vCPUs, 15 GB RAM).
 - Justificación Técnica: Spark requiere memoria sustancial para mantener los DataFrames en caché y evitar el spilling a disco. La familia N1 ofreció un rendimiento de E/S superior para la lectura concurrente desde GCS, reduciendo el tiempo de ejecución del script etl_final_opt.py en un 40% comparado con la configuración E2.

4. Estrategia de datos: Diseño del data lake y gobierno

Para la capa de persistencia y gestión de activos de información, hemos evolucionado más allá del esquema tradicional de Data Warehouse monolítico hacia una arquitectura de Lakehouse simplificada. Esta decisión estratégica sitúa a Google Cloud Storage (GCS) como la columna vertebral de nuestra infraestructura de almacenamiento físico. La elección de GCS no obedece únicamente a criterios de eficiencia de costos (capa de almacenamiento commodity), sino que responde a la necesidad crítica de desacoplar el almacenamiento del cómputo. Al tratar a GCS

como el repositorio inmutable y duradero (con una durabilidad de clase empresarial del 99.999999999%), garantizamos que los datos crudos y procesados persistan independientemente del ciclo de vida de los clústeres de procesamiento Dataproc, los cuales pueden ser terminados para optimizar el gasto (FinOps). Esta separación nos otorga una elasticidad infinita: podemos escalar el volumen de datos a nivel de petabytes sin necesidad de aprovisionar infraestructura de cómputo ociosa, alineando el gasto operativo directamente con la utilidad del dato.

4.1. Estructura de buckets y zonas

El bucket principal `gs://proyectofinalbi2025-2-datalake` se segmentó lógicamente para mantener el orden y facilitar el ciclo de vida de los datos:

- **/bronze (Landing Zone):** Aquí residen los archivos inmutables `bronze_sots.json` y `bronze_contratas.json`. Nadie tiene permisos de escritura aquí excepto la cuenta de servicio de ingestión.
- **/scripts (Code Repository):** Almacenamiento de la lógica de negocio versionada (`etl_sot_v2.py`, `etl_final_opt.py`). Separar el código de los datos es vital para la seguridad.
- **/tmp (Staging Area):** Zona efímera utilizada por el conector de BigQuery (`spark-bigquery-connector`) para escribir datos temporales antes de la carga atómica final.

Esta separación garantiza que, ante un fallo catastrófico en el Warehouse, siempre podemos reconstruir la historia desde la zona `/bronze` (Replayability).

5. Ingeniería de datos: Pipeline ETL con PySpark

El núcleo operacional de nuestra arquitectura de procesamiento reside en el script `etl_final_opt.py`. A diferencia de los pipelines limitados al SQL tradicional, donde la lógica procedimental a menudo se vuelve rígida y difícil de depurar, PySpark nos ofrece un entorno de programación funcional robusto. Esto nos permite realizar validaciones algorítmicas complejas, aplicar transformaciones vectorizadas y gestionar tipos de datos anidados de forma programática. Al trasladar la lógica de calidad de datos a esta capa de cómputo, logramos sanear inconsistencias en memoria antes de impactar la base de datos, protegiendo así la integridad del Data Warehouse contra datos corruptos.

5.1. Análisis del script de transformación

El script implementa una lógica robusta de limpieza y enriquecimiento. A continuación, se detallan los segmentos críticos del código desplegado en el clúster:

A. Lectura y tipado fuerte

No confiamos en la inferencia de esquemas automática para producción. Definimos esquemas o forzamos conversiones explícitas para evitar errores en fechas, un problema común en los logs anteriores.

```
# Fragmento de etl_final_opt.py
# Lectura optimizada desde la zona Bronze
df_sots = spark.read.json("gs://proyectofinalbi2025-2-datalake/bronze/bronze_sots.json")

# Transformación y Limpieza
# Uso de funciones vectorizadas de Spark para rendimiento
df_processed = df_sots.withColumn("fecha_creacion_dt",
to_date(col("fecha_creacion"))) \
    .withColumn("costo_operativo", col("costo").cast("double")) \
    .fillna({"estado": "PENDIENTE", "costo": 0.0})
```

B. Lógica de negocio (Cálculo de KPIs)

Calculamos el tiempo de atención directamente en el ETL para liberar de carga computacional a BigQuery.

```
# Cálculo de latencia operativa (minutos)
df_kpi = df_processed.withColumn(
    "tiempo_atencion_min",
    (unix_timestamp(col("fecha_cierre")) - unix_timestamp(col("fecha_creacion"))) / 60
)
```

C. Escritura eficiente a BigQuery

Utilizamos el modo de escritura overwrite para garantizar idempotencia. Si el job corre dos veces, no duplicamos datos.

Carga a Capa Silver/Gold

```
df_kpi.write.format("bigquery") \
    .option("temporaryGcsBucket", "proyectofinalbi2025-2-datalake/tmp") \
    .option("table", "sot_analytics.stg_sots") \
    .mode("overwrite") \
    .save()
```

6. Modelado dimensional y data warehouse (BigQuery)

Para la consolidación de la capa de persistencia analítica, se procedió con el diseño e implementación de un Modelo Dimensional (Esquema Estrella) alojado dentro del dataset `sot_analytics`. A diferencia de los modelos relacionales altamente normalizados (3NF) que priorizan la eficiencia en escritura, este esquema se ha optimizado específicamente para cargas de trabajo OLAP (Online Analytical Processing). La arquitectura centraliza las métricas cuantitativas del negocio en tablas de hechos y segrega los atributos descriptivos en dimensiones desnormalizadas. Esta estructura reduce la complejidad de las uniones (joins) en tiempo de consulta y maximiza el rendimiento de las operaciones de agregación masiva (SUM, COUNT, AVG), requisitos indispensables para garantizar la interactividad de los tableros de control gerencial.

6.1. Tabla de hechos: `fact_sot`

Esta tabla contiene las métricas numéricas y las claves foráneas.

- Estrategia de Particionamiento:

La decisión arquitectónica más crítica fue particionar la tabla `fact_sot` por el campo `fecha_creacion`.

- Justificación de Costos: BigQuery cobra por bytes escaneados. Al particionar por fecha, cuando un analista consulta "¿Cuántos fraudes hubo en Diciembre?", el motor solo escanea la partición de Diciembre, ignorando el resto del histórico. Esto reduce los costos de consulta drásticamente (hasta un 90% en tablas grandes).

```

/* DDL extraído de los logs de BigQuery */
CREATE OR REPLACE TABLE `proyectofinalbi2025-2.sot_analytics.fact_sot`
PARTITION BY DATE(fecha_creacion)
AS
SELECT
    id_sot,
    id_contrata, -- Foreign Key
    fecha_creacion,
    tiempo_atencion_min,
    costo_operativo,
    CASE WHEN estado = 'FRAUDE' THEN 1 ELSE 0 END as flag_fraude
FROM `proyectofinalbi2025-2.sot_analytics.stg_sots`;

```

6.2. Dimensiones: dim_contrata

Tabla desnormalizada que contiene los atributos descriptivos de las empresas contratistas. Se implementó como una dimensión Tipo 1 (sobrescritura de cambios), asumiendo que el nombre o la zona de la contrata no requieren trazabilidad histórica compleja para este alcance del proyecto.

7. Seguridad, IAM y gestión de costos (FinOps)

La seguridad no fue un añadido, sino un componente base del diseño, evidenciado en la configuración del cluster-blindado-v3.

7.1. Gestión de identidades (IAM)

Se operó bajo el principio de Mínimo Privilegio. No se utilizaron las credenciales de propietario para la ejecución automática.

- **Service** **Account:**
sa-etl-robot@proyectofinalbi2025-2.iam.gserviceaccount.com.
- **Roles Asignados:**
 - Storage Object Viewer: Para leer de /bronze.
 - BigQuery Data Editor: Para escribir en las tablas.
 - Dataproc Worker: Para ejecutar tareas en el clúster.

7.2. Seguridad en aplicación

En los scripts (versión v3), se implementó validación de argumentos mediante `sys.argv` para evitar inyecciones de rutas maliciosas o errores humanos al apuntar a buckets incorrectos.

7.3. Estrategia FinOps (Clusters Efímeros)

Analizando los logs, se observa un patrón de creación y eliminación de clústeres (`dataproc clusters delete`).

- **Modelo de operación:** En lugar de mantener un clúster encendido 24/7 (lo que generaría costos ociosos), adoptamos el modelo de Clústeres Efímeros. El clúster se aprovisiona, ejecuta el `etl_final_opt.py` y se autodestruye. Esto maximiza la eficiencia del gasto computacional, pagando solo por los minutos exactos de procesamiento.

8. Metodología de desarrollo y versionamiento (GitHub)

Para garantizar la colaboración y la trazabilidad del código, simulamos un flujo de trabajo profesional basado en Gitflow.

1. **Rama main:** Contiene únicamente código probado y funcional (Production Ready). Está protegida contra escrituras directas.
2. **Rama feature/etl-fix:** Detectamos un error en el parseo de fechas en la versión 1 del script. Se creó esta rama específica para desarrollar la corrección (`etl_sot_v2.py`).
3. **Pull Request (PR):** Una vez corregido el script localmente y validado con una muestra de datos, se generó un PR hacia main. Esto permitió una revisión de código (Code Review) simulada antes de integrar los cambios.
4. **Despliegue:** Al fusionar (Merge) el PR, el script `etl_final_opt.py` se subió al bucket `/scripts`, asegurando que Dataproc siempre ejecute la última versión aprobada.

9. Conclusiones técnicas

La implementación exitosa del proyecto `proyectorfinalbi2025-2` demuestra la viabilidad y superioridad técnica de una arquitectura Cloud-Native para la inteligencia de negocios moderna.

1. **Escalabilidad:** El paso de máquinas e2 a n1 demostró que la infraestructura es elástica; podemos responder a picos de demanda (ej. Black Friday) simplemente ajustando la configuración del clúster sin comprar hardware físico.
2. **Performance de Consultas:** El particionamiento en BigQuery transformó una tabla de hechos potencialmente lenta en un activo de información ágil, permitiendo a los analistas filtrar millones de SOTs en segundos.
3. **Integridad y Gobierno:** La segregación de zonas en el Data Lake y el uso de Service Accounts dedicadas aseguran que los datos sean auditables, seguros y fiables para la toma de decisiones estratégicas.

Esta arquitectura sienta las bases para futuras implementaciones de Streaming o Machine Learning, posicionando a la organización a la vanguardia tecnológica.